

Parte 1

6	• Se espera modelar el problema utilizando grafos dirigidos en donde los nodos representan los materiales y dispositivos, mientras que las aristas representan las dependencias entre ellos. • Pueden utilizar orden topológico, identificando el uso de grados de entrada. Un dispositivo se puede crear sólo cuando su grado de entrada llega a 0, es decir, cuando todas sus partes están listas. Otra opción es usar DFS donde para crear un dispositivo se parte desde el objetivo y se verifica en profundidad si todas sus partes están disponibles o pueden ser fabricadas. Evaluación: • 3 pts por modelar el problema correctamente. • 3 pts por utilizar un método correcto para recorrer el grafo. Puntaje parcial por explicación incorrecta/falta explicación. En caso de estrategias alternativas, corresponde puntaje completo si se verifica correctitud.
---	---

Parte 2

7	• Debe partir ordenando a los minions se forma descendente según eficiencia. • Se implementa un Min-heap para almacenar a los candidatos según velocidades. La idea es ir agregando hasta llegar a k minions, y luego ir quitando al ingeniero con menor velocidad para agregar al siguiente. Vamos guardando el mejor resultado hasta el momento. • Esta es una estrategia greedy que consiste en fijar la eficiencia mínima del grupo al ordenar. Luego el heap asegura que estamos sumando las mayores velocidades posibles para esa eficiencia. Finalmente, al repetir este proceso para cada ingeniero (usando el heap), estamos probando todos los posibles pisos de eficiencia que podrían existir en la solución óptima, quedándonos con el mejor resultado. Evaluación: • 2 pts por ordenar los minions. • 2 pts por uso de heaps u otra estructura si es bien justificada. • 3 pts por explicar y justificar a grandes rasgos por qué funciona la estrategia. Puntaje total por estrategias alternativas bien justificadas. Puntaje parcial por no justificar o justificaciones incompletas.
---	---

Parte 3

5	• Se esperaba el uso de programación dinámica para resolver el problema. Para esto deberían llevar una tabla M de $n \cdot m$ siendo n el tamaño de la primera palabra d_1 y m el de la segunda d_2, donde: $M[i][j] = \text{ed}(d_1[1...i], d_2[1...j])$ • El problema de edit distance tiene sub-estructura óptima. Es decir, el mejor resultado para cierto paso puede ser calculado con los pasos anteriores. • Por lo tanto, no puede ser resuelto con una estrategia greedy, ya que no siempre se puede seguir la misma estrategia para utilizar sub-problemas anteriores. • Al calcular edit distance es necesario utilizar repetidas veces un mismo sub-problema, por lo que el mejor enfoque es almacenar esta información para no realizar cálculos repetidos. Evaluación: • 1 pt por mencionar el uso de programación dinámica. • 2 pts por explicar que tiene sub-estructura óptima. • 2 pts por explicar que se pueden almacenar y reutilizar valores anteriores. Corresponde puntaje parcial por sólo explicar con palabras o errores menores.
2	• La razón de por qué al implementar bloques se mantienen las propiedades, es que de todas formas se utilizan las mismas operaciones anteriores, con la diferencia de que son representadas de forma distinta. Lo anterior no elimina la propiedad de sub-estructura óptima y la reutilización de cálculos anteriores. Evaluación: • 2 pts por explicar por que la sub-optimalidad se mantiene al implementar bloques.